

4차 산업혁명에 맞서는 IoT, 음식물 종량 처리 시스템

-슈렉이-

SeongEun Oh, NaRae Lee, SeungHee Heo

School of Computer Engineering, Dongyang Mirae University, xxx@dongyang.ac.kr

School of Computer Engineering, Dongyang Mirae University, xxx@dongyang.ac.kr

School of Computer Engineering, Dongyang Mirae University, xxx@dongyang.ac.kr

Abstract 본 연구는 일상에서 들리는 미확인 음악을 신속히 식별하고 그 결과를 기기 간에 이어서 활용할 수 있도록, ACRCLOUD 기반 인식 결과를 앱-웹-백엔드로 연결한 일체형 파이프라인(DMUSIC)을 설계·구현하였다. Android 앱은 오디오 샘플을 ACRCLOUD REST API로 전송하고 반환 메타데이터를 표준화하여 저장한다. 서버/웹은 Supabase(PostgreSQL)와 연동해 곡 정보를 조회·관리하며, Spotify/YouTube API를 통해 연관 탐색을 제공한다. 파일럿 과업을 통해 엔드투엔드 처리시간(녹음 시작부터 결과 제시까지 총 소요 시간), TOP-1 매칭률(첫 제시 곡의 정답률), 조작 단계 수 (인식→저장→웹 확인까지 요구되는 클릭 수)를 관찰하여 실용성을 검토하고, 호텔·카페 등 환대산업에서의 현장 적용 가능성을 논의한다.

Keywords: RFID, IoT, 오픈 API, Python

1. 서론

1. 연구 배경 및 문제 정의

스마트폰과 스트리밍 플랫폼의 확산으로 음악 소비 경험은 '탐색-발견-감상'의 연속적 형태로 재구성되었다. 이에 따라 사용자들은 일상에서 접하는 배경음악을 즉시 식별하고 관련 메타데이터(제목, 아티스트, 앨범 등)를 확인한 뒤, 재생목록에 저장하거나 관련 영상을 탐색하기를 원한다. 그러나 기존의 상용 음악 인식 서비스들은 오디오 인식, 메타데이터 관리, 플레이리스트 연동 기능이 서로 다른 애플리케이션 및 계정 체계로 분절되어 있다. 또한, 흥얼거림(허밍)과 같은 비정형 입력에 대한 인식 품질 및 모바일과 웹 간의 동기화 일관성 확보에도 한계를 보인다. 따라서 이러한 한계를 극복하고 매끄러운 연동이

가능한 음악 인식-수집-공유 파이프라인을 설계하고 그 실용성을 검증할 필요가 있다.

2. 연구의 목적, 기술적 의의 및 논문 구성

이러한 문제의식 하에, 본 연구는 음악 인식 결과를 앱-웹 간에 끊김 없이 연동하는 일체형 파이프라인 DMUSIC을 설계하고 그 실용성을 검증하는 것을 목적으로 한다.

이를 위한 구체적인 목표는 다음과 같다.

본 연구는 ACRCLOUD, Supabase 등 여러 기술 스택을 통합하여 효율적인 음악 데이터 연동 구조를 제시하고 그 성능을 검증한다는 점에서 기술적 의의를 갖는다.

논문은 다음과 같이 구성된다. 제2장에서는 관련 이론 및 선행 연구를 검토하고, 제3장에서는 DMUSIC의 아키텍처 설계와 구현 내용을 기술한다. 제4장에서는 시스템 검증 결과를 제시하고 논의하며, 제5장에서는 결론과 향후 연구를 제언한다.

II. 관련 이론 및 기술적 배경(DMUSIC)

2.1 음악 인식 기술 및 오디오 핑거프린팅 (ACRCLOUD)

음악 인식(Music Identification)은 미확인된 오디오 샘플을 데이터베이스에 저장된 방대한 음원과 비교하여 원본 정보를 찾아내는 기술이다. 본 연구는 상용 API인 ACRCLOUD를 핵심 인식 엔진으로 사용한다.

2.1.1 오디오 핑거프린팅 원리

오디오 핑거프린팅(Audio Fingerprinting)은 음악 파일의 특징점을 추출하여 고유한 디지털 "지문"을 생성하고, 이를 데이터베이스에 저장된 지문과 대조하여 음원을 식별하는 기술이다. 이는 소음, 압축, 시간적 변형 등 환경적인 변화에도 강인한(robust) 특징을 추출하여 높은 인식률을 유지한다. ACRCLOUD는 이러한 핑거프린팅 기술을 기반으로 오디오 샘플을 비교하여 메타데이터(제목, 아티스트 등)를 반환한다.

2.1.2 ACRCLOUD API의 역할 및 특징

ACRCLOUD는 방대한 음원 라이브러리를 기

반으로RESTful API를 제공하며, 본 시스템은Android 앱에서 캡처한 오디오 샘플을 이 API로 전송하여 실시간으로 음악 정보를 얻는다. 이API는 높은 정확도와 빠른 응답 속도를 제공하여DMusic 시스템의 신속한 인식 목표를 달성하는 핵심 요소로 작용한다.

2.2 실시간 데이터 연동 및 관리 기술 (Supabase)

DMusic 시스템은 인식된 음악 기록의 앱-웹간 실시간 동기화를 위해Supabase 플랫폼을 백엔드 서버로 활용한다.

2.2.1 Supabase의 구조 및 특징

Supabase는PostgreSQL 기반의 오픈 소스 서버리스 플랫폼으로, 인증(Authentication), 데이터베이스(Database), 실시간(Realtime) API를 통합 제공한다. 본 연구에서는 특히 PostgreSQL의 안정성과 Realtime API기능을 활용하여, Android 앱에서DB에 기록된 음악 데이터를 웹 인터페이스에서 즉시 조회할 수 있도록 한다.

2.2.2 JSON 데이터 파싱 및 표준화

ACRCloud API가 반환하는 원본 메타데이터는 복잡한JSON 형태로 구성되어 있다. 시스템의 효율적인 데이터 처리 및 저장을 위해, 이JSON 데이터 중 핵심 정보(곡명, 아티스트, 앨범 등)만을 추출하여 DMusic 시스템 표준 형식으로 파싱하고Supabase 테이블에 저장한다. 이러한 표준화 과정은 이후Spotify나YouTube와 같은 외부API와의 연동을 용이하게 한다.

2.3 외부API 연동 기술(Spotify 및YouTube)

DMusic은 사용자가 인식된 음악 정보를 바탕으로 추가적인 탐색 및 감상을 할 수 있도록Spotify 및YouTube API를 활용한다.

III. DMusic 시스템 설계 및 구현

3.1 시스템 아키텍처 및 데이터 흐름

DMusic 시스템은 기존 상용 서비스의 분절성을 해소하고 음악 인식 결과를 앱-웹 간에 끊김 없이(seamless) 연동하기 위해 3계층 아키텍처로 설계되었다. 시스템은 클라이언트 레이어(Android App, Web Interface), 서비스 레이어(ACRCloud, Spotify, YouTube API), 그리고**데이터/백엔드 레이어(Supabase)**로 구성된다.

3.1.1 시스템 아키텍처 구성

본 시스템은 Supabase를**데이터의 단일 소스(Single Source of Truth)**로 활용하여 모든 데이터의 영속적인 저장과 실시간 동기화를 담당한다. 주요 구성 요소와 그 역할은 다음과 같다.

3.1.2 사용자 인증 및 데이터 저장 흐름

DMusic 시스템은Supabase의 인증(Authentication) 기능을 활용하여 앱과 웹

에서 동일한 계정 체계를 공유한다. 사용자가 로그인할 때 획득하는**사용자 고유 식별자(User ID)**는 모든 인식 기록 및 플레이리스트 데이터에 연결되어 저장된다.

3.1.3 핵심 기능별 데이터 처리 파이프라인

DMusic의 핵심 가치인 앱-웹 연동은 다음과 같은 두 가지 파이프라인을 통해 구현된다.

A. 실시간 음악 인식 및 기록 저장 파이프라인(앱 중심)

B. 웹 기반 실시간 동기화 및 탐색 파이프라인(웹 중심)

3.2 데이터베이스(Supabase) 및 메타데이터 설계

DMusic 시스템은PostgreSQL 기반의 서버리스 플랫폼인Supabase를 백엔드로 활용하여 사용자 인증, 인식 기록의 저장 및 앱-웹간의 실시간 데이터 동기화를 위한 단일 데이터 소스(Single Source of Truth)를 구축하였다.

3.2.1 핵심 테이블 구조 및 관계

시스템은 사용자 관리, 인식 기록, 그리고 플레이리스트 기능 구현을 위해 다음과 같이3가지 핵심 테이블을 포함하여 총10개의 테이블로 설계되었다. 모든 테이블은user 테이블의**user_code**를**참조 키(Foreign Key, FK)**로 사용하여 사용자별 데이터의 독립성을 확보한다.

1. user 테이블(사용자 정보)

사용자 인증 및 식별을 위한 기본 테이블이다.

Name	Description	Data Type	Format	Nullable	
user_code	No description	bigint	int8	X	⋮
user_id	No description	text	text	X	⋮
user_pw	No description	text	text	X	⋮
user_name	No description	text	text	✓	⋮
google_key	No description	text	text	✓	⋮
kakao_key	No description	text	text	✓	⋮

2. acr 테이블(음악 인식 기록)

사용자가 앱에서 인식한 곡의 메타데이터를 표준화하여 저장하는 핵심 테이블이다.

Name	Description	Data Type	Format	Nullable	
artist	No description	text	text	X	1
album	No description	text	text	X	1
label	No description	text	text	✓	1
release_date	No description	date	date	✓	1
score	No description	double precision	float8	✓	1
scr_title	No description	text	text	✓	1
user_code	사용자가 국영방송 관제망에 supabase에 user_code를 넘겨야 하기 위함	bigint	text	X	1
album_image_url	No description	text	text	✓	1

3. playlist_track 테이블(플레이리스트 목록)
플레이리스트와 그 안에 담긴 곡을 연결하는 중간 테이블이다.

Name	Description	Data Type	Format	Nullable	
user_code	No description	bigint	text	X	...
playlist_id	No description	text	text	X	...
added_at	No description	timestamp without time zone	timestamp	✓	...
track_name	No description	text	text	✓	...
artist_name	No description	text	text	✓	...
image_url	No description	text	text	✓	...
track_id	No description	text	text	✓	...

3.2.2 ACRCLOUD 메타데이터 표준화

ACRCLOUD API가 반환하는JSON 응답은 다양한 필드를 포함하므로, DMusic 시스템은 이를 데이터베이스에 저장하기 위해 표준화 과정을 거친다.

3.2.3 실시간 동기화 아키텍처(Supabase 활용)

DMusic의 핵심인 앱-웹 연동은Supabase의 Realtime 기능과Spring Boot 백엔드의 통신 게이트웨이를 통해 달성된다.

3.3 사용자 인터페이스(UI) 및 핵심 기능 구현 상세

3.3.1 Android 앱: 오디오 샘플링 및 ACRCLOUD 연동 구현

DMusic의Android 앱은 사용자에게 직관적인 음악 인식 경험을 제공하며, ACRCLOUD API와의 연동을 통해 실시간에 가까운 결과를 얻는다. 앱은 크게 오디오 샘플링, ACRCLOUD 요청 구성, 그리고 데이터 표준화 및 백엔드 저장의 세 단계를 거쳐 기능을 수행한다.

1. 오디오 샘플링 및 녹음 관리

앱의 주 활동인MainActivity는 사용자의 마이크를 통해 오디오를 캡처한다.

2. ACRCLOUD 요청 구성 및 전송

녹음이 중지되면stopRecording() 함수 내에서AcrCloudRepository를 호출하여 인식 요청을 시작한다. 이 과정은ACRCLOUD API가 요구하는 서명(Signature) 기반 인증을 통해 이루어진다.

3. ACRCLOUD 응답 처리 및 데이터 표준화

AcrCloudRepository 내의 비동기 코루틴(CoroutineScope(Dispatchers.IO).launch) 블록에서ACRCLOUD 서버의 응답을 수신하고 처리한다.

이 파이프라인은 오디오 샘플링부터DB 저장까지의 과정을 끊임 없이처리하여, 시스템의 핵심 목표인 신속한 인식과 기록을 달성한다.

3.3.2 웹 인터페이스: 인식 기록 조회 및 시각화 구현

DMusic의 웹 인터페이스는 Android 앱에서 수집된 인식 기록을 사용자가 시각적으로 확인하고, 통계 정보를 파악할 수 있는 대시보드 역할을 수행한다. 이 과정은 단순한 데이터 조회를 넘어, 비동기 처리를 통한 메타데이터 보강(Enrichment)과 시각화, 그리고 외부 플랫폼 연동을 포함한다.

1. 인식 기록 조회 및 메타데이터 보강 (Backend Logic) 웹 인터페이스의 요청을 처리하는 백엔드 서버는 MyPageController를 중심으로 Raw 데이터 조회 → 외부 API 기반 정보 확장 → 병합의 3단계 파이프라인을 통해 데이터를 제공한다.

ACR 기록 조회: 클라이언트로부터 전달받은 userCode를 키(Key)로 사용하여, AcrService를 통해 Supabase에 저장된 1차 인식 기록(곡 제목, 아티스트명)을 조회한다.

비동기 메타데이터 보강 (Enrichment): Supabase에 저장된 데이터는 텍스트 위주의 경량 데이터이므로, 시각화를 위한 앨범 커버 이미지나 미리듣기 URL 등의 정보가 부재하다. 이를 보완하기 위해 시스템은 조회된 각 레코드에 대해 SpotifyService를 호출하여 상세 정보를 요청한다. 이때 다수의 트랙에 대한 외부 API 호출 지연(Latency)을 최소화하기 위해 Java Reactor(Flux/Mono)를 활용, 모든 요청을 병렬로 처리(Flux.merge)하여 성능 저하를 방지하였다.

데이터 병합 및 반환: 병렬로 수집된 상세 정

보들은 하나의 TrackDetailDto 리스트로 비동기적으로 취합(collectList)되어 클라이언트에 반환된다.

2. 반응형 UI 렌더링 및 통계 시각화 (Frontend Logic) 웹 클라이언트는 React 프레임워크를 기반으로 구현되었으며, 서버로부터 수신한 JSON 데이터를 사용자 친화적인 형태로 가공하여 렌더링한다.

데이터 동기화 및 바인딩: 불필요한 네트워크 트래픽을 방지하기 위해, 클라이언트는 화면 진입 시점(Mount)과 사용자의 명시적 요청(Refresh) 시에만 데이터를 동기화하는 상태 기반 업데이트(State-based Update) 방식을 채택하였다. fetchRecommendations 비동기 함수를 통해 수신된 데이터는 map() 함수를 통해 UI 컴포넌트와 바인딩되어, 사용자는 앨범 아트가 포함된 카드 형태의 리스트로 자신의 검색 기록을 직관적으로 확인할 수 있다.

클라이언트 사이드 데이터 집계 및 시각화: 단순 나열을 넘어 사용자의 음악 취향을 시각화하기 위해, 클라이언트 내부에서 데이터를 재가공하는 로직을 수행한다. 수신된 전체 트랙 리스트를 순회하며 아티스트별 인식 빈도를 집계(Aggregation)하고, 상위 10개 항목을 추출한다. 이 데이터는 Chart.js 라이브러리와 연동되어 원형 차트(Pie Chart)로 렌더링됨으로써, 사용자가 가장 많이 인식한 아티스트의 분포를 한눈에 파악할 수 있도록 구현하였다.

3. 외부 API 연동 및 확장 탐색 기능 (External Integration) 웹 인터페이스는 인식된 곡 정보를 기반으로 Spotify 및 YouTube API를 활용하여 미디어 경험을 확장하고 영구적으로 기록을 관리한다.

Spotify 기반 트렌드 및 검색: 메인 대시보드(MainPage.jsx)는 Promise.all을 사용하여 Spotify의 신곡 리스트와 YouTube 인기 차트 데이터를 병렬로 호출함으로써 지연 없는 로딩을 구현하였다. 또한 검색 기능 사용 시 Spotify 카탈로그를 조회하여 정확도 높은 곡 정보를 제공한다.

YouTube 미디어 임베딩: 사용자가 상세 페이지에 진입하면, 시스템은 곡명과 아티스트명을 조합하여 YouTube API를 호출한다. 검색된 최상위 결과(Top-1)인 공식 뮤직비디오나 커버 영상을 페이지 내 iframe으로 즉시 임베딩(Embedding)하여, 페이지 이탈 없는 감상 환경을 구축하였다.

영구적 플레이리스트 관리: 사용자는 탐색한 곡을 개인 플레이리스트에 추가하거나 삭제할 수 있다. 생성된 플레이리스트 데이터는 Spring Boot 백엔드를 통해 Supabase의 playlist_track 테이블에 저장되어 영구적으로 관리된다.

3.4 배출량 기록

인증된 사용자가 ‘슈렉이’에 음식물을 버리면 자동으로 무게센서를 통하여 배출량이 측정된다.

```
sprintf(query, "insert into history
(r_id,weight,pay,year,month,day,hour,min,sec)
values('%s',%s,%s,%s,%s,%s,%s,%s,%s)",
row[0],data,cost,year,mon,mday,hour,mini,sec);
/* db에 값 등록 */
mysql_query(mysql, query);
```

[그림 10] 배출량 저장 SQL

[그림 10]의 SQL문을 사용하여 HISTORY 테이블에 사용자의 ID(r_id)와 배출량, 무게, 시간이 [그림 11]과 같이 저장된다.

no	r_id	weight	pay	year	month	day	hour	min	sec
1	A101	100	1000	2017	10	21	8	10	11
2	A101	55	550	2017	10	23	7	50	12
3	B101	78	780	2017	10	25	9	40	47
4	A101	170	170	2017	10	27	7	43	20
5	A101	25	250	2017	10	27	18	30	34
6	B101	250	2500	2017	10	31	8	15	11

[그림 11] HISTORY 테이블의 데이터

3.5 데이터 확인

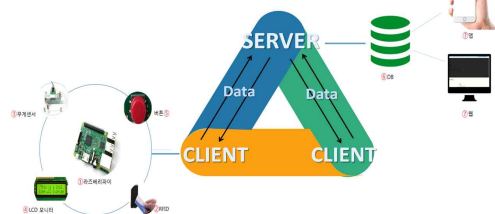
사용자는 웹과 앱에서 ID와 PASSWORD를 입력하면 자신의 배출량을 조회하여 그래프와 표로 실시간 관리가 가능하다. 이러한 기능을 통해 처리비용 감소 효과를 기대한다.

2. 구현

4.1 프로그램 동작 과정



[그림 12] 웹/앱을 통한 데이터 확인



[그림 13] 프로그램 동작 과정

‘슈렉이’는 라즈베리파이를 통해 센서를 제어한다. 무선AP를 통해 서버와 클라이언트가 자동으로 실행되며 Thread를 사용하여 여러 대의 클라이언트에서 다중접속이 가능하다. ①RFID를 태그하면 사용자가 인식되고, 서버모터를 이용하여 자동으로 뚜껑의 덮개가 열린다. ②배출량의 무게를 측정한다. ③측정된 무게와 금액이 모니터에 출력이 된다. ④버튼을 누르면 덮개가 닫히고 데이터가 서버로 전송된다. ⑤음성합성을 통해 무게 값과 금액이 스피커로 출력되어 사용자에게 안내한다. ⑥측정된 무게 값과 금액, 날짜, 시간이 데이터베이스에 저장된다. ⑦사용자는 실시간으로 앱과 웹에서 사용량 조회가 가능하다.

4.2 구현환경

라즈베리파이를 기반으로 하여 Python언어와 C언어를 사용해 각각의 서버와 클라이언트를 구축하였다. 데이터베이스는 MySQL을 이용하였다. 안드로이드에서 JSP서버로 데이터베이스 접근을 시도하고 JSON을 파싱하여 결과 값을 출력하였다.

3. 결론

기존에 음식물 처리로 지불한 비용을 확인할 수 없었던 점을 보완하여 IoT 장비와 RFID기술을 이용한 종량 처리 시스템을 개발하였고, 데이터베이스를 사용해 실시간으로 관리가 가능하며 사용자로 하여금 정보를 한눈에 파악할 수 있도록 하였다. 앞으로는 일정 시간 동안 무게센서의 값이 유지되도록 동작하는 것과 안드로이드 UI를 수정하여 데이터 확인 기능의 장점을 더 돋보일 수 있도록 개선할 계획이다.

감사의 글

프로젝트에 앞서 저희 조를 잘 이끌어 주시고 많은 조언과 도움을 주신 정석용 교수님께 감사합니다.

참고문헌

- [1] <http://cs.sch.ac.kr/lecture/CompApp/RFID/p13524.pdf>
- [2] <http://blog.naver.com/rllrkcka/220612235200>
- [3] <http://blog.naver.com/chandong83/220956890937>

오성은 (라즈베리)	이나래 (안드로이드)	허승희 (웹)